

Agent IA pour le Support Informatique

1. Démarche suivie

Le projet vise à construire un agent intelligent spécialisé dans le support informatique (gestion des mots de passe, messagerie, réseau, VPN, imprimantes et accès aux applications), en appliquant une approche **Agentic RAG** : le système ne se limite pas à rechercher des informations dans une base documentaire, mais laisse le LLM raisonner, planifier et décider dynamiquement quels outils utiliser.

Les étapes suivies :

- **Construction de la base documentaire** : création de documents PDF contenant des procédures de dépannage informatique (mot de passe, réseau, VPN, messagerie, imprimante, accès aux applications), puis indexation avec **PyPDFLoader**, découpage avec **RecursiveCharacterTextSplitter** et vectorisation avec **HuggingFaceEmbeddings** (modèle *sentence-transformers/all-MiniLM-L6-v2*) dans un **InMemoryVectorStore**.
 - **Développement des outils** : création de quatre outils (**search_kb**, **search_web**, **diagnose**, **create_ticket**) permettant la recherche documentaire, la recherche Web, le diagnostic de pannes et la création d'un ticket de support.
 - **Développement du graphe LangGraph** : création d'un état, d'un nœud agent (LLM + outils) et d'un nœud d'exécution des outils reliés par des transitions conditionnelles.
 - **Ajout de la mémoire** : utilisation de **InMemorySaver** afin de conserver le contexte de la conversation grâce au **thread_id**.
 - **Visualisation du graphe** avec **draw_mermaid_png** et **évaluation** du système sur 20 questions (10 simples et 10 complexes).
-

2. Fonctionnement du système

2.1 Architecture modulaire

Le projet est organisé en plusieurs modules indépendants :

- **state.py** : état partagé de l'agent.
 - **prompts.py** : System Prompt.
 - **rag.py** : construction de la base documentaire.
 - **tools.py** : outils utilisés par le LLM.
 - **graph.py** : construction du graphe LangGraph.
 - **test.py** : tests automatiques de l'agent.
-

2.2 Boucle agentique (le graphe)

Le graphe commence par **START**, puis le LLM analyse la question de l'utilisateur dans le nœud **agent**. Selon le besoin, il répond directement ou appelle un ou plusieurs outils. Le nœud **tools** exécute les outils demandés puis renvoie leurs résultats au LLM sous forme de **ToolMessage**. Ce cycle se répète jusqu'à ce que le LLM fournisse une réponse finale, puis le graphe atteint **END**.

2.3 Logique de décision RAG / Web

L'agent consulte d'abord la base documentaire interne grâce à **search_kb**. Si les informations sont insuffisantes ou si la question concerne une information récente (mise à jour logicielle, nouvelle version d'un logiciel, actualité informatique), il utilise **search_web**.

Lorsque l'utilisateur décrit un problème technique (réseau, imprimante, VPN, messagerie ou mot de passe), l'agent utilise **diagnose** pour proposer une procédure de dépannage. Si le problème persiste, il peut créer automatiquement un ticket grâce à **create_ticket**.

2.4 État et mémoire

L'état **HelpDeskAgentState** conserve l'historique de la conversation, le nombre d'appels au LLM, le nombre d'appels aux outils ainsi que les sources consultées. La mémoire **InMemorySaver**, associée à un **thread_id**, permet de conserver le contexte de la conversation pendant toute la session.

2.5 Visualisation

Le graphe est généré automatiquement au lancement de **test.py** sous la forme d'une image **graph.png**.

3. Résultats obtenus de l'évaluation

Le script **evaluation.py** exécute automatiquement **10 questions simples** (questions couvertes par la base documentaire) et **10 questions complexes** (diagnostic, combinaison de plusieurs outils, création de ticket ou recherche Web).

Pour chaque question sont mesurés :

- le temps de réponse ;
- le nombre d'appels au LLM ;
- le nombre d'appels aux outils ;
- les sources utilisées.

Les résultats sont enregistrés automatiquement dans un fichier Excel nommé **test.xlsx**.

4. Limites et pistes d'amélioration

4.1 Limites identifiées

- Base documentaire limitée aux procédures présentes dans les PDF.
 - Mémoire temporaire avec **InMemorySaver**.
 - Modèle local (**Llama 3.2:3B**) offrant des capacités de raisonnement limitées par rapport à des modèles plus puissants.
 - Les outils ne sont pas connectés à un véritable logiciel de gestion de tickets ou à un annuaire d'entreprise.
 - Les tickets générés sont simulés et ne sont pas enregistrés dans un système réel.
 - Les performances du RAG dépendent directement de la qualité et de la mise à jour des documents utilisés.
-

4.2 Pistes d'amélioration

- Enrichir la base documentaire avec davantage de procédures informatiques.
 - Utiliser une base vectorielle et une mémoire persistantes.
 - Connecter l'agent à un véritable outil de gestion des tickets (GLPI, Jira Service Management, ServiceNow, etc.).
 - Développer une interface Web pour les utilisateurs.
-

Conclusion

En conclusion, **HelpDeskBot** montre l'intérêt de combiner **LangGraph**, le **RAG**, les **outils** et la **mémoire** afin de développer un assistant intelligent spécialisé dans le support informatique. Ce projet met en œuvre les principaux concepts des agents IA et constitue une base solide pour développer des assistants capables d'automatiser les demandes de support technique et d'améliorer l'assistance aux utilisateurs.